

蓝桥杯30天算法冲刺集训



Day-12 数位DP

所谓的数位DP就是求解的对于任意的一个范围 $[L, R]$ 内，所有满足条件 f 的数的个数，也就是求解：

$$ans = \sum_{i=L}^R f(i)$$

如果朴素的去求的话我们需要用类似如下的代码假设 f 条件判断需要使用时间 $O(m)$ 遍历 $[L, R]$ 需要使用时间 $O(n)$ ，那么下面代码的整体复杂度是 $O(nm)$ 的，对于大数据来说类似 $1 \leq L \leq R \leq 10^9$ ，显然是要超时的

```
for(int i=L;i<=R;i++)  
    if(check(i)) ans++;
```

那么我们就需要使用相应的优化方式，因为数位DP理解起来对于初学者难度较大，所以本讲义用一个引例，一个例题，一个蓝桥杯国赛真题，一个练习题来带大家理解数位DP真正的含义。数位DP只要理解了，那么运用起来确比较公式话，状态的定义都基本大同小异，唯一不同的就是状态之间的转移，需要自己再考虑一下。而其他的DP类似状压DP，状态的定义就已经百花齐放，所以说数位DP是上手困难但是用起来简单的DP。

⚠ Caution

本讲义所提到的数位DP都是类似上面公式的，用来记录数字类型的DP，广义上讲，只要对数位操作的DP都可以称作数位DP。但是本讲义并不包含后者，后者的话可以用很多的DP技巧，例如贪心暴力搜索背包等等，并不局限于记录数字，所以我感觉这种问题可以归类于其他的DP。本讲义提到的记数类型的DP却具有独特性。

引例

📖 Note

下面的问题只需要理解题意即可，给出的示例代码可以参考也可以直接跳过，后面会给出本讲义统一的数位DP模拟过程。

按位求和问题

题面

给定 l, r , 求 $[l, r]$ 内的所有数的 k 进制表示下各数位之和
 $1 \leq l \leq r \leq 10^5, 2 \leq k \leq 10$

分析

我们先从我们比较熟悉的十进制入手。

直接一个一个枚举, 花费的时间是不可估量的, 所以我们要充分利用它的一些性质来帮助我们计算。由于我们求的是某东西的和, 根据和的性质, 我们可以把它拆成 $[0, r]$ 这个东西的和减去 $[0, l-1]$ 这个东西的和。

接着, 我们思考这个问题的一个简化的形式, 就是 r 不是任意给出的, 而是 $10^t - 1$ 这一类的数。为什么我们要取这样一个特殊的数呢? 因为当我们选了这样的一类数后, 我们的每个数位都可以从 0 取到 9 了。

现在我们以 99 为例, 来算 $[0, 99]$ 内的所有数的 10 进制表示下各数位的和。

对于十位, 我们可以选 $0 \sim 9$ 的任何数, 对于个位, 我们可以选 $0 \sim 9$ 的任何数。于是, 我们可以通过这种方式来算各个数位的和: 对于十位为 $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$ 的情况, 都各自有 10 种情况 (也就是个位为 $0, 1, \dots, 9$), 对于个位为 $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$ 的情况, 也各自有 10 种情况, 因此 $0 \sim 99$ 各个数位的和为 900。

同理, 我们也可以算出 $[0, 999]$ 各数位的和 (为 13500, 请同学们自行计算), 以及任意多个 9 的情况答案的和。

现在我们用 $f(t)$ 表示 t 个 9 对应的答案, 求出 $f(t)$ 的时间是 $O(\log t)$ 。

那问题是, 现在的 r 或者 $l-1$ 并不是这个形式, 我求出上面那个答案有什么用呢? 实际上, 我们可以这样转化问题, 将问题转化成上面那个问题:

以 r (或者 $l-1$) = 5987 为例, 我们看看是怎么转化的。

对于 $[0, 5987]$ 我们可以通过分类讨论, 将问题分成很多上面的这个子问题。先讨论千位, 千位只可能是 $0, 1, 2, 3, 4, 5$ 五种情况, 如果是前 4 种情况, 那么后面三位, 就可以取到 $[0, 999]$ 的所有情况, 由于我们确定了第一位, 于是后面三位就是前面我们求出的那个子问题, 比如当千位为 3 时, $[3000, 3999]$ 的数位和就等于 $3 * 1000 + f(3)$ (千位的数字 $\times$ 后面三位的总方案数 + 百十个位的情况)。因此, 能求出千位为 $0, 1, 2, 3, 4$ 对应的答案就等于 $10 * 1000 + 5f(3)$ 。这只需要 $O(k)$ 的时间, k 是进制, 这里取 10 因为我们已经计算了千位为 $0, 1, 2, 3, 4$ 的情况, 千位只剩下 5 的情况没有取了。我们令千位为 5, 这时候, 要考虑的就是百位了, 百位在千位为 5 的情况下, 有 $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$ 十种情况, 这时候, 如果百位取前九种情

况，十位和个位都能取到 0,99。于是我们就可以把答案又轻易地表示出来了，以 4 为例，这时候我们确定前两位为 54，因此答案就等于 $9 * 100 + f(2)$ 。

这样我们又计算出了百位为 0,1,2,3,4,5,6,7,8 的情况，只剩下百位为 9 的情况没算了，我们令百位为 9，这时候前两位就是 59，十位只能是 0,1,2,3,4,5,6,7,8 这些情况，如果是前八种情况，个位就能取到 [0,9]，以十位为 6 为例，这时候确定前三位为 546，答案就等于 $15 * 10 + f(1)$ 。最后我们令十位为 8，个位就只能取 0,1,2,3,4,5,6,7 了，由于个位后面没有位了，所以直接计算个位的答案即可，比如个位为 6 时，答案为 $23 * 1 + f(0)$ ， $f(0)$ 也就是 0，个位为 7 时，答案为 $24 * 1 + f(0)$ 。

将这个过程所有答案都加起来，就是总答案了。我们来计算一下时间复杂度，应该是 $O(\text{位数} * k)$ 的，位数就等于 $\log_{10} r$ 或者 $\log_{10}(l-1)$ ，相比枚举是很大的提升。

刚才我们只算了 $k = 10$ 的情况，其他进制也是类似的，只不过我们需要把 9，对应成 $k-1$ ，在开始计算前，将 r 和 $l-1$ 转化成 k 进制数，存在数组里面。

```
int data[18],len; //将r或者l-1的数位存在数组中，从左到右存,比如
456,data[0]=6,data[1]=5,data[2]=4,len=3
int f[18];
long long ans = 0;
void count(int nowlength,int pre){ //使用时，传入count(2,0),pre
表示前面的数位
    if(nowlength == -1){
        ans += pre;
    }else{
        for(int i=0;i<data[nowlength];i++){
            ans += 1ll*
(pre+i)*power_k[nowlength]+f[nowlength]; //k^nowlength
        }
        count(nowlength,pre+data[nowlength]); // pre*k就是左移，
再补上当前位
    }
}
void prepare(){
    for(int i=1;i<len;i++){
        f[i] = 1ll*i*(k-1)*k/2*power_k[i-1];
    }
}
```

```
}
```

例题

通过上面的讲解，我们可以得知数位DP实际上就是遍历两种情况，假设我们现在取的数是第 t 位的。

一种就是 $10^t - 1$ 的情况，也就是说我们当前可以取到 $[0, 10^t - 1]$ 内任意的数，那么我们也可以把这个状态保存下来，后面再用到的话还是同样的计数原理，直接做加和即可。

另一种就是我们不能随便取数，取数有限制，例如我们到了目前数位的数字的最大值，后面的数并不是能正好取到 $[0, 10^t - 1]$ 这个范围的，那么我们需要再额外的去计算一下。

我们去模拟这个过程的时候呢，我推荐使用**记忆化搜索**，因为我们都是递归回溯再把相应的值加在一起的，所以记忆化搜索是比较合适的。

最终我们的复杂度近似是 $O(k)$ ， k 代表数字的长度。

小蓝怕4

通过题目我们可以得知，我们需要的是数位里面不带有4的数字的个数，那么我们可以直接进行记忆化搜索，分上面所说的两种情况。

我们可以定义一个 $f[i]$ 代表数字从后到前长度为 i ，并且满足数字在给定的数字范围之内，所具有符合规定的数字的个数

具体代码示例如下：

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 16;
int a[N];
ll f[N];
ll dfs(int pos,int lim)
{
    if(pos==0) return 1;
    if(!lim && f[pos]!= -1) return f[pos];
```

```

    int up = lim?a[pos]:9;
    ll ans=0;
    for(int i=0;i<=up;i++)
    {
        if(i==4) continue;
        ans += dfs(pos-1,lim&& i==a[pos]);
    }
    if(!lim) f[pos]=ans;
    return ans;
}
ll sum(int x)
{
    int pos=0;
    while(x)
    {
        a[++pos]=x%10;
        x/=10;
    }
    return dfs(pos,1);
}
int main()
{
    int a,b;
    while(cin>>a>>b)
    {
        if(a==0&&b==0) break;
        memset(f,-1,sizeof(f));
        cout<<sum(b)-sum(a-1)<<"\n";
    }
}

```

首先我们看 main函数 里面的代码

```

int main()
{
    int a,b;
    while(cin>>a>>b)
    {
        if(a==0&&b==0) break;
        memset(f,-1,sizeof(f));
        cout<<sum(b)-sum(a-1)<<"\n";
    }
}

```

这份代码表示的是我们输入了 a, b 代表区间左右的端点，然后我们初始化 f 数组，接着我们调用 `sum` 函数 来计算相应的数字个数也就是 $[0, b]$ 和 $[0, a-1]$ 之间满足 f 的数字的个数，那么这个差值就是最终的答案。

我们再来看 `sum`函数

```

ll sum(int x)
{
    int pos=0;
    while(x)
    {
        a[++pos]=x%10;
        x/=10;
    }
    return dfs(pos,1);
}

```

这个函数实际上是把给定的整数按位拆分然后再放进我们的记忆化搜索里面的，同学们可能会问为什么要这么做，我们直接输入一个 `string` 不也可以吗？

因为我们的两个整数 a, b 不一定是同样的位数的，所以我们这样倒着拆分可以写一个记忆化搜索，因为返回的条件都是 `pos==0`，但是如果用正着走的输入两个 `string` 的话，不知道两个字符串的具体长度，再加上输入的字符串里面的数字都是 `char` 类型的ASCII码，还需要进一步的把它变成整数。所以模拟数位DP会略显麻烦。当然如果同学们觉得正着直接输入字符串比较合适，也可以用正着输入字符串来模拟数位DP。

好的下面就让我们看看我们今天的重头戏，也就是记忆化搜索吧

```

ll dfs(int pos,int lim)
{
    if(pos==0) return 1;
    if(!lim && f[pos]!= -1) return f[pos];
    int up = lim?a[pos]:9;
    ll ans=0;
    for(int i=0;i<=up;i++)
    {
        if(i==4) continue;
        ans += dfs(pos-1,lim&&i==a[pos]);
    }
    if(!lim) f[pos]=ans;
    return ans;
}

```

首先看第一行

```

ll dfs(int pos,int lim)

```

这一行定义了两个参数，一个是 `pos` 代表当前的位数，一个是 `lim` 代表当前的位数是不是没有限制的，因为就是我们上面说的，肯定要分两种情况，一种是没有限制的，可以取到 $[0, 10^t - 1]$ 那么我们就可以计算并且记录下来后面也是重复的状态，另一种就是有限制的类似12345的第二位数字，如果我们求的是11xxx那么就是无限制的，可以后面任意的取数，也就是说可以取到 $[0, 999]$ 之间任意的数。但是我们如果是12xxx那就是有限制的，后面最多只能取到345。

然后看后面两行

```

    if(pos==0) return 1;
    if(!lim && f[pos]!= -1) return f[pos];

```

这两行定义了搜索返回的条件，也就是说要么我们 `pos==0` 代表我们当前选的数是合适的，直接返回1，要么我们现在可以任意的取，没有限制并且已经记忆化了，就直接返回记忆化后的数组即可。

然后看下面的几行


```
int up = lim?a[pos]:9;
ll ans=0;
for(int i=0;i<=up;i++)
{
    if(i==4) continue;
    ans += dfs(pos-1,lim&& i==a[pos]);
}
```

这里的 `up` 代表的是我们当前数位能取到的最大的数字，如果没有限制显然就是 9，如果有限制显然就是当前数位上面给定的数。

接着我们再遍历当前数位上面的数字。

发现如果当前数位上面有4的话，显然不能往下进行计算，如果没有4的话，那么接着计算即可

然后再看最后两行

```
if(!lim) f[pos]=ans;
return ans;
```

也就是看看当前的状态是不是不加限制的，如果是的话，就记录一下，最后返回我们当前走的这个状态的值。

蓝桥杯真题

二进制问题（蓝桥杯C/C++2021B组国赛）

不知道同学们看到这题感受如何呢？是不是有被吓到呢（*^_^*），实际上如果前面理解的话，这题也难度不大的，就是改改我们的转移就好了。

首先我们看到需要恰好为 k 个1，那么是不是跟上面有没有4是一个类型的问题？也就是我们需要额外的记录一下当前已经有几个1了，也就是说如果最后到达 `pos==0` 的时候，如果当前走过的1的个数恰好为 k 个，那么就返回1，否则返回0。

然后我们需要记忆化我们的状态，我们是不是可以记录一下当前可以随便选的情况下恰好能选 k' 个1的个数？

那么我们就可以设置 $f[i][j]$ 代表二进制从后往前长度为 i 位二进制的个数恰好为 j 的二进制数的个数，然后进行转移就好了。

怎么转移呢？实际上是不是还是一样的，我们主要的循环就是在找合法的情况，次要的就是在记录没有限制情况下 $f[i][j]$ 的值，跟上面的例题是同样的道理的。

时间复杂度大概是 $O(n)$ ，代表的是二进制的数位长度，二进制最多也不过64位。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 80;
ll f[N][N], a[N];
ll dfs(ll pos, ll lim, ll cnt, ll maxn)
{
    if(pos==0)
    {
        if(cnt==maxn) return 1;
        else return 0;
    }
    if(!lim && f[pos][maxn-cnt]!=-1) return f[pos][maxn-cnt];
    int up=lim?a[pos]:1;
    ll ans=0;
    for(int i=0; i<=up; i++)
    {
        if(cnt==maxn&&i==1) continue;
        ans+=dfs(pos-1, lim&&i==a[pos], cnt+(i==1), maxn);
    }
    if(!lim) f[pos][maxn-cnt]=ans;
    return ans;
}
ll sum(ll x, ll y)
{
    int pos=0;
    memset(f, -1, sizeof(f));
    while(x)
    {
        a[++pos]=x%2;
```

```

        x/=2;
    }
    return dfs(pos,1,0,y);
}
int main()
{
    ll x,y;
    cin>>x>>y;
    cout<<sum(x,y);
}

```

习题

数字计数

详细题解

我们还是把问题转化成求 $[0, b]$ 的答案减去 $[0, a - 1]$ 的答案。

枚举 b (或 $a - 1$) 的数位，我们要排除前导零的影响，因此，我们发现 0 的地位很特殊，也就是说，比如 $b = 5430$ 时，我们对于它的千位，不仅要递归首位为 5 的情况，还要讨论首位为 0 的情况，因为这时候有前导 0。

对于千位，我们可以取 0, 1, 2, 3, 4, 5。如果取 0，问题会变成：对于 $[0, 999]$ ，前导零不算的情况下，每个数码各出现了几次。如果取 1, 2, 3, 4，问题会变为，对于 $[000, 999]$ ，前导零也算的情况下，每个数码出现了多少次，同时还要加上 1, 2, 3, 4 出现 1000 次。如果千位取 5，那么就需要考虑百位了。

我们考虑百位的时候，0 还需要特殊看待吗？不用，因为千位已经确定是 5 了，因此不存在所谓的前导 0 对于百位，我们有 0, 1, 2, 3, 4 这些取法，当取 0, 1, 2, 3 时，问题会变为对于 $[00, 99]$ ，前导零也算的情况下，每个数码出现了多少次，还要加上 0, 1, 2, 3 出现 100 次。如果百位取 4，那就相当于确定了前两位为 54，就考虑十位了。

后面的过程和取百位类似，就不再重复了。

现在我们分析一下，我们需要预处理的实际上就是， $[0, 10^t - 1]$ 这些数中，算前导零的情况下 (固定 t 位数)，每个数位出现了多少次。以及 $[0, 10^{b.length-1}]$ 这些数中，不算前导零的情况下，每个数字出现了多少次。

对于前者，我们和上一道题的处理方法相同，很容易写出每个数字出现的次数就等于 $t * 10^{t-1}$ 。对于后者，我们还是要分成首位是 0 和首位不是 0 去求，当首位是 0 时，它变成了一个子问题： $[0, 10^{b.length-1} - 1]$ 中，不算前导零的情况下，每个数码出现了几次，求解子问题，我们很容易想到 DP，假设这个已经用 $f[b.length - 1]$ 表示出来了，那么我们只需要 $f[b.length] += f[b.length - 1]$ 。当首位不是 0 时，它后面就可以任取了，我们利用已经预处理出的算前导零的答案，来推出这时候的答案。

实现过程

我们还是要分两种情况讨论，一种是无限制的选取，一种是有限制的，这里要记录数位的个数，那么需要对于有限制的选取和无限制的选取都需要额外的进行判断，加上后面的数字个数。

还有就是前导 0 情况，需要单独的分开讨论，我们可以看到如果当前数字前面都是连续的前导 0 那么我们这一位还是选 0 的话会继续前导 0，如果不是的话前导 0 就不受影响了，于是我们可以使用 $pre \mid i$ 操作来模拟这个是不是前面已经出现一个非 0 的数了。

剩下的就是跑 10 次数位 DP 即可。

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 16;
int a[N];
ll f[N][10][N], base[N], ot[N], maxn, now;
ll dfs(int pos, int lim, int pre, int digit)
{
    if(pos==0) return 0;
    if(!lim && f[pos][digit][pre] != -1) return f[pos][digit][pre];
    int up = lim?a[pos]:9;
    ll ans=0;
    for(int i=0; i<=up; i++)
    {
        if(i==digit)
        {
            if(pre==0 && i==0);
            else if(lim && i==up) ans += (now % base[pos-1]) + 1;
        }
    }
}
```

```

        else ans += base[pos-1];
    }
    int tmp =pre|i;
    tmp = min(tmp,1);
    ans += dfs(pos-1,lim&&i==a[pos],tmp,digit);
}
if(!lim) f[pos][digit][pre]=ans;
return ans;
}
void sum(ll x,int sign)
{
    int pos=0;
    now=x;
    memset(f,-1,sizeof(f));
    while(x)
    {
        a[++pos]=x%10;
        x/=10;
    }
    maxn=pos;
    for(int i=0;i<=9;i++)
    {
        if(sign==0) ot[i] += dfs(pos,1,0,i);
        else ot[i] -= dfs(pos,1,0,i);
    }
}
int main()
{
    base[0]=1;
    for(int i=1;i<N;i++) base[i]=base[i-1]*10;
    ll a,b;
    cin>>a>>b;
    sum(b,0);
    sum(a-1,1);
    for(int i=0;i<=9;i++) cout<<ot[i]<<" ";
}

```